LOCAL VERIFICATION FOR CODING AGENTS

InvariantGuard

Complete build manifest

A local-first, change-scoped verifier that combines modeled runtime-failure analysis, repository-specific behavioral contracts, local open-weight proposal and concrete evidence in the coding-agent repair loop.

18 MARKDOWN FILES	55 IMPLEMENTATION TASKS	8 MILESTONES
Root Claude instructions	Dependency-ordered backlog	Python to Kotlin to expansion

Build the first real vertical slice: source -> typed IR -> static obligation -> witness -> structured CLI result. Then add patch impact, behavioral replay, local semantic discovery and MCP.

Version 1.0 | 4 October 2026

The manifest reproduces the complete Markdown specification. Example outputs and performance targets are design proposals. Nothing in this document claims an already implemented product or universal program correctness.

Contents

Read the root handoff first. Technical specifications follow in build order; the final section is the implementation checklist.

InvariantGuard build kit 3
Claude implementation instructions 6
Start building with Claude 8
01. Product and scope 10
02. Architecture 13
03. Evidence and policy 15
04. Verification IR and language adapters 19
05. Static engine 22
06. Contracts and specification discovery 25
07. Incremental analysis and baselines 28
08. Local open-weight reasoning 30
09. Interfaces and storage 32
10. Execution boundary and privacy 36
11. Roadmap and release gates 38
12. Testing, benchmarks and research 41
13. Demo and acceptance scenarios 44
14. Decisions, risks and reading 47
15. Ordered implementation backlog 50

InvariantGuard build kit

README.md

Version 1.0 | 4 October 2026 | Implementation specification, not an implemented product

InvariantGuard is a local-first, change-scoped verification layer for coding agents. It learns repository-specific behavioral contract candidates, checks relevant contracts and modeled runtime-failure paths, and returns evidence into an agent repair loop. Open-weight inference proposes semantic properties; mechanical analysis and explicitly enabled execution establish the evidence.

Start here

1. Put this entire folder at the root of the repository you want Claude to build.
2. Keep `CLAUDE.md` at that root. Keep these specifications in `docs/`.
3. Start Claude in the folder and paste the opening task from `BUILD_START.md`.
4. Build milestone M0, then M1. Use `docs/15_TASK_BACKLOG.md` as the checklist.
5. Read the PDF manifest for the same complete plan in a reviewable document.

This kit contains planning documents only. Example commands describe the intended interface; they do not imply a CLI is already installed. The PDF is a rendering of every Markdown file, including this README, the agent instructions and the implementation backlog.

File map

File	Purpose
CLAUDE.md	Root-level implementation instructions and hard evidence rules
BUILD_START.md	Copyable first task, follow-up tasks and first-session definition of done
docs/01_PRODUCT_AND_SCOPE.md	Product thesis, users, release scope and boundaries
docs/02_ARCHITECTURE.md	Modules, data flow, repository layout and dependencies
docs/03_EVIDENCE_AND_POLICY.md	Proof, observation, provenance and gate semantics
docs/04_VERIFICATION_IR.md	Shared IR, explicit language semantics and frontend contract
docs/05_STATIC_ENGINE.md	Supported Python subset, analyses and solver behavior
docs/06_CONTRACT_ENGINE.md	Candidate discovery, typed property DSL and lifecycle
docs/07_INCREMENTAL_AND_BASELINES.md	Snapshots, impact analysis, invalidation and comparison
docs/08_LOCAL_MODELS.md	Open-weight reasoning, offline operation and model protocol
docs/09_INTERFACES_AND_STORAGE.md	CLI, MCP, report examples, configuration and persistence
docs/10_EXECUTION_AND_PRIVACY.md	Execution boundary, sandbox behavior and data handling
docs/11_ROADMAP_AND_GATES.md	Dependency-ordered milestones and release criteria
docs/12_TESTING_AND_RESEARCH.md	Semantic fixtures, benchmarks, ablations and reproducibility
docs/13_DEMO_AND_ACCEPTANCE.md	Exact demo fixtures and acceptance scenarios
docs/14_DECISIONS_AND_REFERENCES.md	Decision record, risks and primary-source reading
docs/15_TASK_BACKLOG.md	Ordered implementation tasks with concrete completion checks

Build sequence

M0 foundation -> M1 Python static CLI -> M2 patch impact and baselines -> M3 behavioral contracts -> M4 local semantic discovery -> M5 MCP -> M6 Kotlin -> M7 broader language and library adapters.

The first integrated product milestone is M5. Python static checks alone are an engineering checkpoint. They do not fulfill the whole project. Kotlin remains part of the planned beta; broader language support follows only after semantic adapter tests exist.

Important interpretation

All timings, model sizes, supported-language counts and quality targets are proposed goals unless explicitly labeled as measured. No name, package identifier, trademark or license choice is finalized by this kit. No prior research code is included or assumed available.

A successful scan means selected checks completed without a blocking finding. It never means the entire repository is correct. An observed regularity remains a candidate until its evidence and specification authority are recorded.

Claude implementation instructions

CLAUDE.md

You are implementing InvariantGuard from the specifications in this folder. Read `README.md`, `BUILD_START.md`, `docs/03_EVIDENCE_AND_POLICY.md`, `docs/11_ROADMAP_AND_GATES.md` and the tasks for the active milestone before changing code. Read the relevant technical specification before building its module.

Product definition

Build a local-first, incremental verifier for coding-agent patches with two coordinated engines: modeled runtime-failure analysis and repository-specific behavioral contract verification. Add a local open-weight model as a contract proposer. Keep CLI and MCP results machine-readable so another coding agent can repair a patch from the returned evidence.

Work rules

- Implement in dependency order. Begin with M0 and a real M1 vertical slice.
- Choose Python for the orchestration/core prototype and first frontend. Implement Kotlin second through a versioned adapter. Do not start by building eight parsers.
- Use a minimal typed Verification IR from the beginning. Extend it only when a shipped analysis needs another operation. Do not build a compiler optimization framework.
- Preserve actual language semantics. Operators sharing syntax can have different failures. Unsupported behavior must propagate uncertainty.
- An LLM response, a type annotation, a passing sample or a solver model of an approximate program is never sufficient for an unconditional proof.
- Keep proof status, execution evidence, coverage, authority and severity separate. Follow the schemas and gate rules.
- Use real source analysis and real test execution when enabled. Do not hard-code the demonstration findings or use regex as the final Python frontend.
- Implement the no-model path. Make open-weight reasoning a real optional runtime capability, not a stub advertised as complete.
- Use a typed property DSL. Do not `eval` candidate strings or execute arbitrary model-produced code.
- Static commands must never import the repository, install dependencies, run its hooks, or execute its tests.
- Dynamic checks require the explicit run configuration described in the execution specification. A subprocess is not an isolation sandbox.
- Capture immutable snapshots for every verification. Abort comparison or mark it incomplete if the repository changes during the run.
- Track cross-file impact. An unchanged caller can become unsafe when a callee changes.
- Intentional API changes can revise approved contracts through an explicit, auditable change. Never automatically accept a changed baseline just to silence a failure.
- Never change the repository being checked. Never auto-repair user code, reset branches or discard uncommitted files.
- Keep network use out of verification by default. Loopback inference is allowed only for a configured local endpoint.
- No model downloads during scans. No cloud fallback. No telemetry by default.

- Pin tested dependencies in a lock file; record solver, interpreter and adapter versions. Use documentation for the exact versions selected.
- Update task checkboxes only when their acceptance checks pass. Leave unsupported features documented and unimplemented rather than inventing evidence.
- Use small reviewable changes. Run checks appropriate to each change. Record commands and outcomes in `docs/IMPLEMENTATION_STATUS.md`.

Evidence requirements

Use `PROVED_SAFE` and `PROVED_UNSAFE` only for the declared property, modeled semantics, quantifier, assumptions and scope. A bounded search must say bounded. A satisfiable overapproximation yields a possible failure, not a proven real path. Passing property tests yields `TESTED`, not proof.

A observed regression must include the same witness run against the baseline and patch, environment identities, and a confirmed baseline pass plus patch failure. If baseline already fails, label it pre-existing. If baseline does not run, comparison is unknown.

A scan with missing required coverage is `INCOMPLETE`, even if no defects were found. An engine crash is `ERROR`. Neither becomes `PASS`.

First implementation target

Build the two `ratio` examples in `docs/13_DEMO_AND_ACCEPTANCE.md` end to end: AST -> typed IR -> interval/path reasoning -> optional solver -> real source location -> stable JSON -> CLI exit code. Include the safe guard case and the unsupported-call case.

Do not implement local models, dynamic importing, Kotlin, a web dashboard or large language support before this path works.

Definition of done for any milestone

Working implementation, meaningful semantic fixtures, no fabricated outputs, documented supported subset, accurate uncertainty propagation, repeatable commands, and an updated status file. If a prerequisite is unavailable, implement the independent parts and record the concrete remaining blocker.

Start building with Claude

BUILD_START.md

Paste this as the first task

Read CLAUDE.md and the specifications it names. Implement M0 and M1 of InvariantGuard in this repository. Start by stating the supported Python subset and the module boundaries. Create the package, typed result and IR models, AST frontend, interval/path analyzer, optional Z3 integration, and a working static CLI.

Make the guarded and unguarded ratio fixtures from docs/13_DEMO_AND_ACCEPTANCE.md work through actual analysis. Add meaningful tests for branch reachability, unsupported semantics, source mapping and JSON/exit behavior. Static checks must not import or execute the target repository. Report incomplete coverage honestly.

Run the relevant checks, update docs/IMPLEMENTATION_STATUS.md, and leave a runnable Python static-check vertical slice. Stop at the M1 acceptance gate with the implementation status and exact commands. Do not simulate future features or claim the full product is finished.

First-session decisions

Use Python 3.11 or newer for the host package; select and document one tested interpreter version. Use stdlib AST for Python parsing, dataclasses plus strict serialization/schema validation, argparse for the first CLI, pytest for tests, and Z3 as an optional solver dependency. Choose exact dependency versions during implementation and lock them.

Keep `guard` as the desired local executable name. Use `invariantguard` as the source package. Check public package and command naming before any public release; the working name is provisional.

Treat the specification as version 1.0. If details conflict, follow evidence and language semantics rules first, then record a small decision with the rationale. Do not silently redesign the whole system.

M1 first-session completion

- `guard check PATH --format json` analyzes supported Python without importing it.
- Unsafe ratio includes a satisfiable exact modeled path and an integer witness.
- Guarded ratio proves the division obligation safe in its declared integer domain.
- Unknown call handling demonstrates honest incomplete coverage.
- JSON is schema-valid and stable; diagnostics go to stderr.
- Tests run on CPU with no network or local model.
- An engine failure and an unsupported operation are different outcomes.
- The status file identifies every implemented and deferred capability.

Follow-up tasks

M2

Implement immutable base/current snapshots, patch-to-symbol mapping, conservative dependency invalidation, summary caching and differential static comparison. Include changed-callee/unchanged-caller, untracked files, deletions and rename fixtures. Measure warm and cold runs separately.

M3

Implement the typed contract DSL, immutable evidence records, baseline/candidate authority states and an explicitly enabled isolated probe runner. Reproduce a true normalization regression on the same valid input in both snapshots. Keep finite tests distinct from proofs.

M4

Implement the local model adapter protocol and candidate schema. Integrate one real open-weight model hosted on the developer's machine, with bounded context and no external traffic. Mechanically check every proposed property. Record model identity and test invalid/adversarial suggestions.

M5

Implement a stdio MCP server over the same orchestration API as the CLI. Demonstrate verify -> evidence -> agent repair -> verify with an actual client configuration. Return coverage and assumptions, not only a verdict.

M6

Integrate Kotlin through a typed adapter, initially targeting a documented Kotlin/JVM compiler version. Validate integer division, array bounds and null assertions against isolated compiled fixtures. Reuse the core report, policy and contract schemas.

Progress record

Create `docs/IMPLEMENTATION_STATUS.md` during M0 with: active milestone, supported semantics, completed tasks, validation commands, measured results, known gaps and next task. Do not fill it with speculative results.

01. Product and scope

docs/01_PRODUCT_AND_SCOPE.md

Thesis

A local-first verification layer for coding agents incrementally analyzes generated patches, discovers repository-specific behavioral contract candidates, checks modeled runtime-failure paths and contract changes, and returns concrete evidence directly into the repair loop.

The product primitive is `verify(base, candidate, policy) -> report`. A coding agent can call it repeatedly while editing. The user receives a task completion claim supported by bounded verification evidence rather than a code review opinion alone.

Users and jobs

User	Immediate job	Useful output
Developer using an agent	Catch newly introduced failures before accepting a patch	Source span, path, witness and repair context
ML/data researcher	Preserve numeric, shape, schema and determinism properties	Valid-domain counterexample and baseline comparison
Maintainer	Keep repository-specific API behavior across refactors	Contract provenance and intentional-change workflow
Privacy-sensitive team	Verify without sending repository contents to a cloud model	Offline analysis and auditable local inference

The initial user is a developer running an agent on a small Python project. Research and ML examples are useful demonstration cases, but the core is not specific to PLForecast, sports simulation or one repository.

Product principles

Local operation is the default. Open-weight reasoning is a first-class layer with a pluggable adapter and a no-model fallback. Repository-specific evidence complements generic runtime-failure classes. The report distinguishes exact modeled proof, observed failure, likely issue, candidate property and unknown coverage. Patch comparison attributes differences without pretending all observed behavior is intended.

Scope ladder

Checkpoint	Real capability	Deliberate boundary
M1	Python static failure checks over a declared subset	Single snapshot; no learned dynamic contracts
M2	Change impact, caching, before/after static differences	Conservative invalidation; no universal equivalence proof
M3	DSL contracts and isolated differential probes	Execution must be configured; tests are finite evidence
M4	Local model proposes semantic candidates	Model cannot approve contracts or certify safety
M5	CLI/MCP agent repair integration	No claim of universal agent compatibility
M6	Kotlin adapter and shared checks	Version-pinned JVM semantics initially
M7	TS and selected library adapters, then other frontends	Coverage grows only after semantic conformance tests

The full planned project includes Python, Kotlin, TypeScript and eventually additional languages, a shared verification core, local open-weight inference and agent integrations. M1 is a narrow step toward that project, not a replacement definition.

Initial bug and contract classes

M1: division-by-zero feasibility for supported integer domains; basic reachability; explicit unknowns. M2 adds constant/interval index bounds and narrow None receiver flows. M3 adds nonempty-input normalization, numeric range, selected argument immutability and repeatability contracts. Kotlin beta targets integer division, array bounds and explicit null-assertion failures. Tensor shapes, DataFrame columns and JSON schemas require versioned library summaries and arrive later.

Resource leaks, concurrency races, arbitrary casts, exception-policy inference, network availability and whole-program equivalence are later research directions. No deadline is promised for them.

Success criteria

A useful finding gives a property or failure class, affected scope, exact source span, evidence class, declared assumptions, a witness when available, baseline relationship, and a reproducible replay recipe when execution is enabled.

A useful successful report says which obligations were checked, which were safe within their assumptions, which contracts were tested, which checks were skipped and whether policy permits accepting the patch.

The first integrated release must include a real model-assisted contract discovery path, a no-model path, change-aware verification, evidence persistence and an actual CLI/MCP repair demonstration.

Intent and intentional changes

The old implementation is evidence of prior behavior, not an oracle of desired behavior. Every learned contract has an authority state. Auto-mined properties default to advisory. A developer can accept a deliberate behavior change by revising an approved contract with a reason and linked patch. The verifier checks the revised contract separately; it never silently redefines success to match the patch.

Performance goals

Target a warm static path below two seconds at p95 on a defined reference laptop and small-to-medium fixtures. Treat sub-second cases as a stretch goal. Use separate budgets for solver checks, replay and model inference. Publish hardware, workload, coverage and cold/warm measurements. No latency figure in this specification is a measured result.

02. Architecture

docs/02_ARCHITECTURE.md

End-to-end pipeline

1. Resolve repository root and explicit base/current snapshot identities.
2. Read source bytes without importing repository modules.
3. Parse supported files and lower typed, source-mapped Verification IR.
4. Map changes to symbols, dependencies, contracts and library models.
5. Compute a conservative affected slice and invalidate stale summaries.
6. Run cheap static transfer rules; escalate supported uncertain obligations to SMT within budgets.
7. Retrieve approved and advisory contracts; optionally propose new candidates through local inference.
8. Mechanically check DSL properties statically where possible and dynamically only when execution is enabled.
9. Compare baseline/candidate evidence on the same scope and witnesses.
10. Persist immutable evidence, evaluate gate policy and return structured results.

Contract discovery can run as a background or explicit task. A quick verification reads existing contracts without invoking a model. Re-discovery must not block every agent edit.

Proposed source tree

```
src/invariantguard/
  cli.py
  orchestration.py
  models/          # versioned reports, evidence, contracts and source spans
  snapshots/       # immutable source manifests; optional Git reader
  frontends/
    python/        # AST, symbol resolution, lowering and semantics profile
    kotlin/        # future version-pinned typed adapter
  ir/              # typed expressions, control-flow and effects
  analysis/        # intervals, null states, local flow and summaries
  solver/          # supported obligation encodings and budgets
  contracts/       # DSL validation, discovery, lifecycle and checking
  impact/          # symbol/dependency graph and invalidation
  execution/       # explicit runners and isolation capability checks
  semantic/        # local model protocol and structured candidates
  storage/         # SQLite metadata and content-addressed artifacts
  policy/          # finding classification and verdict aggregation
  mcp_server.py    # optional extra; shared orchestration entrypoints
tests/
  unit/
  semantic_fixtures/
  integration/
  differential/
benchmarks/
  fixtures/
  manifests/
examples/
  python_demo/
  kotlin_demo/
docs/
```

Use interfaces between modules rather than making an LLM adapter call the solver or runner directly. Only orchestration coordinates evidence production and applies policy.

Internal interfaces

```
Frontend.lower(source_snapshot) -> ModuleIR + Coverage
Analyzer.check(module_ir, scope, summaries) -> ObligationResults
Impact.plan(base_graph, candidate_graph, changes) -> AnalysisPlan
ContractEngine.validate(candidate) -> TypedContract or Rejection
ContractEngine.check(contract, snapshot, plan) -> Evidence
SemanticAdapter.propose(context_packet) -> CandidateBatch
Runner.replay(probe, snapshot, run_profile) -> ReplayResult
Store.append_evidence(evidence) -> ContentId
Policy.evaluate(results, coverage, policy) -> Verdict
```

Each call carries a run ID, cancellation token, resource budget and schema/engine identity where relevant. Errors are typed. A runner timeout is not a program failure.

Technology choices

Start with Python orchestration, stdlib AST, dataclasses, explicit JSON schemas, SQLite and optional Z3. Use pytest for semantic and integration tests. Add property-based tooling only when dynamic contracts are implemented. Use the official MCP SDK as an optional dependency after the CLI API stabilizes.

Do not select a default local model from an unverified name or size mentioned in earlier brainstorming. In M4, evaluate available open-weight models for structured contract generation on the actual development hardware, pin one tested artifact and license, and document it. Model integration should support a configurable loopback endpoint.

Kotlin adapter choices require a small feasibility spike: compiler FIR/IR extension versus a typed sidecar using supported compiler APIs. Compiler internals can change. Prefer a reproducible pinned sidecar boundary; do not import an existing research plugin without access, compatibility and license review.

Tree-sitter is optional later for fast syntax indexing. It does not replace type resolution, CFG construction or language semantics.

Reuse and separation

Use existing parsers, compiler frontends, solver and property-testing libraries. Build the integration logic: patch scope, contract provenance, evidence types, typed candidate validation, policy and agent-facing counterexamples.

An external analyzer can contribute findings through an adapter but cannot bypass evidence classification. Tool output retains its version, rule ID, trust level and precision limits.

Dependency boundaries

The core should import neither MCP nor model clients at startup. `guard check` must work without a model, network, dynamic runner or compiler sidecar. Missing optional dependencies produce a capability status and actionable error only when the user requests that capability.

Atomicity and concurrency

Multiple runs may read snapshots concurrently. Use SQLite transactions for metadata and immutable artifact IDs for results. Finalize a run only after its expected evidence is committed. Invalidate unfinished runs after interruption; do not reuse them as passing cache entries. Never have background discovery mutate the authority state of approved contracts.

03. Evidence and policy

docs/03_EVIDENCE_AND_POLICY.md

Separate the axes

Do not make one numerical confidence value stand in for proof, frequency, intent and severity. Every result carries these independent axes:

Axis	Values or meaning
property_result	PROVED_SAFE, PROVED_UNSAFE, POSSIBLY_UNSAFE, TESTED, OBSERVED_FAILURE, OBSERVED_REGRESSION, UNVERIFIED, UNKNOWN
scope_validity	EXACT_MODEL, CONDITIONAL, OVERAPPROXIMATE, BOUNDED, EXECUTED
authority	DECLARED, APPROVED, INFERRED, REJECTED, RETIRED
patch_relation	INTRODUCED, PRE_EXISTING, RESOLVED, CHANGED, UNDETERMINED
severity	ERROR, WARNING, INFO, according to declared policy
coverage	supported obligations, unsupported constructs and skipped work
run_outcome	COMPLETE, INCOMPLETE, ERROR, CANCELLED
verdict	PASS, WARN, BLOCK, INCOMPLETE, ERROR

A proof is a statement about one property under explicit assumptions. A gate verdict is a policy decision about a whole run.

Exact meaning of result labels

PROVED_SAFE: the bad-state obligation is unsatisfiable in a sound supported model, within declared inputs, semantics and effects. Quantify the property and scope. This can prove only division safety for a function, not all safety or total correctness.

PROVED_UNSAFE: an existential bad-state path is satisfiable and the modeled execution is exact for that obligation. Emit the concrete witness and assumptions. This means a failure is possible for at least one admitted input, not that every invocation fails. An unvalidated hypothetical caller path cannot become an unconditional repository failure.

POSSIBLY_UNSAFE: a bad state is not excluded by an approximate model, external analyzer or incomplete path. A SAT result on an overapproximation may be spurious. State the imprecision.

TESTED: finite probes passed. Record counts, input domains, seeds, environments and failure-search strategy. Never equate no found counterexample with universal truth.

OBSERVED_FAILURE: an enabled execution produced a real failure. Record input and runner result. A timeout, isolation failure or import error in an incompatible environment is not automatically a defect.

OBSERVED_REGRESSION: the same valid-domain witness satisfies the baseline property and violates it in the patch under comparable pinned environments. Record both executions and property evaluation. For nondeterministic results record repeats and uncertainty; do not infer universal determinism from a handful of trials.

UNVERIFIED: a valid candidate exists but has no sufficient mechanical evidence.

UNKNOWN: unsupported semantics, solver unknown, incomplete scope, invalid assumptions or missing environment prevents establishing a result.

Proof obligations and assumptions

A type annotation in Python is a proposed domain restriction, not runtime enforcement. A proof over annotated integer inputs must explicitly say it is conditional on exact built-in integers. For a caller using `isinstance`, consider bool and subclasses; exact built-in semantics may require `type(x) is int` or a declared trusted boundary.

Facts derived only from samples or a local model may not become proof assumptions silently. Distinguish declared preconditions, sound static facts, test observations and guesses. If an operation can call user code through overloads, its numeric rule is conditional or unknown.

For an SMT safety query use `path_conditions AND error_condition`. UNSAT is useful only when the path model and domain cover the specified scope soundly. SAT establishes modeled feasibility only when the relevant modeled path is exact; otherwise it is a possible issue until independently validated.

Bounded unrolling can establish a witness within its modeled bound. A no-failure result within that bound must remain explicitly BOUNDED; it cannot prove safety of an arbitrary loop. In the first release unsupported loops return UNKNOWN instead of being silently truncated.

Evidence record minimums

Every evidence item stores schema version, engine version, run ID, snapshot ID, symbol ID, property ID, result label, domain, assumptions, precision, source span and creation time. Solver evidence additionally stores solver version, query digest, response, optional witness and limits. Execution evidence stores run profile digest, interpreter/compiler identity, environment lock digest, inputs, outputs/exceptions, isolation capability and replay artifacts.

Immutable evidence can be superseded by a newer result but must not be rewritten. Properties and source identities can change; history references the exact revision checked.

Default gate policy

Condition	Default action
Introduced exact modeled runtime failure under validated or explicitly admitted domain	BLOCK
Introduced conditional failure whose preconditions are unconfirmed at real callers	WARN; offer witness validation
SAT on overapproximate model	WARN; never label as proof
Replayed regression of DECLARED or APPROVED contract	BLOCK
Change to INFERRED property	WARN; ask maintainer to review intent through contract workflow
Pre-existing defect with no demonstrated widening	Report separately; no new patch block by default
Pre-existing defect with newly feasible path or broader violating domain	CHANGED finding; evaluate the changed portion
Required obligation unsupported, skipped or timed out	INCOMPLETE
Optional analysis unavailable	WARN plus capability disclosure
Engine/internal failure	ERROR
Required work completed, no blocking findings	PASS or WARN based on advisories

An approved contract regression can be intentionally accepted by the user through contract revision, not by deleting evidence. A static failure in a supposedly required input domain is not waived by silently narrowing the domain.

Aggregation

Verdict precedence: ERROR, BLOCK, INCOMPLETE, WARN, PASS. Preserve run_outcome separately, so a BLOCK report can also be incomplete. CLI BLOCK takes precedence over incomplete but includes both reasons. Required coverage must be defined in the run plan before analysis begins; do not redefine it afterward to manufacture PASS.

Examples

`ratio(a, b)` has an exact integer-model witness `a=1, b=1` for a zero denominator. With only annotations, scope_validity is CONDITIONAL and callers must be evaluated before a repository-level unconditional claim.

A model proposes `output >= 0`; 5,000 probes pass. Result is TESTED, authority INFERRED, quantifier finite sampled inputs. It remains advisory.

A model proposes a probability bound, but a caller passes arbitrary plugin objects. Without modeled overloads the static result is UNKNOWN. The proposal can still be saved as UNVERIFIED.

No field called `confidence: 0.998` is needed. Measured precision belongs in aggregate evaluation reports with denominators and uncertainty, not as a fake posterior on one contract.

04. Verification IR and language adapters

docs/04_VERIFICATION_IR.md

Goal

Reuse analysis structure across languages without erasing semantic differences. The IR is a compact, typed, source-mapped representation of verification obligations and effects. It is not a universal claim that identically spelled operators behave identically.

Minimal v1 objects

Object	Required fields
Module	schema_version, language, semantics_profile, source_digest, symbols, imports, coverage
Function	symbol_id, parameters, declared_domain, blocks, entry_id, exits, effects, source_span
Block	block_id, instructions, terminator, predecessor IDs
Instruction	op, operands, result_id, semantic_type, source_span, effect
Expression	kind, typed operands, literal value or symbol reference
Type	language type, numeric semantics, nullability and certainty origin
SourceSpan	relative path, start/end line and column, byte offsets and snapshot ID
Unsupported	original span, reason, dependent values and effects
Summary	preconditions, result facts, possible exceptions, effects and validity keys

Use immutable IDs within a snapshot. Semantic identity for matching symbols across snapshots is separate from source line numbers. Lower simple assignments to versioned value IDs and joins to phi-like merges when needed; do not force full SSA infrastructure before the first checks work.

Initial instruction vocabulary

`CONST`, `PARAM`, `ASSIGN`, `ADD`, `SUB`, `MUL`, `DIV`, `FLOORDIV`, `MOD`, `COMPARE`, `IS_NONE`, `LEN`, `INDEX`, `CALL`, `ATTRIBUTE`, `RETURN`, `RAISE`, `BRANCH`, `JUMP`, `PHI`, `UNKNOWN_EFFECT`.

Each operation includes a semantics profile or explicit operation flavor. `DIV` cannot mean generic mathematical division for every frontend. For M1, it need only capture the zero-denominator condition for exact built-in integer operands; it must not claim accurate floating-point results.

Semantic profiles

Case	Required handling
Python built-in integer arithmetic	Unbounded integer addition/subtraction; numeric overload uncertainty tracked
Python `/`, `//`, `%`	Built-in zero divisor raises; floor/modulo rules differ from truncation
Python sequence index	Negative indices are valid within bounds; slicing has different behavior
Kotlin/JVM `Int`	Fixed-width arithmetic; overflow behavior modeled, not mathematical integers
Kotlin nullable values	Smart casts, `!!`, Java platform types and declared types have distinct precision
JavaScript Number division	Zero denominator can yield Infinity or NaN; do not emit generic division exception
JavaScript BigInt division	Distinct integer semantics and zero-divisor error behavior
TypeScript array access	Out-of-range access can yield undefined; subsequent use is the relevant failure
Rust/C/C++	Adapter-specific panic, wrapping, undefined behavior and build-mode profiles required

The Kotlin and JavaScript rows are architectural requirements for later frontends. They are not M1 implemented capabilities.

Unsupported behavior

Dynamic dispatch, reflection, native calls, unknown imports, decorators, unsupported exception handling or mutation can invalidate relevant facts. Lower these to unknown results and effect summaries. Taint downstream obligations that depend on them. Report exactly why the property is unresolved; do not drop the function from the denominator.

An unknown call may mutate shared state, throw, call back into the repository or return an arbitrary value. A trusted library summary can narrow that effect, but its version and authority must be explicit.

Frontend adapter protocol

An adapter advertises languages/versions, supported syntax, supported type domains, effect precision, library summaries and compiler requirements. It emits versioned IR plus diagnostics and coverage. The core rejects incompatible IR versions. Parsing success is not semantic-analysis coverage.

Python frontend initially uses AST without imports. Kotlin sidecar uses typed compiler information in an explicitly configured environment. Later frontends can use compiler APIs or parsers, but must pass the same shared adapter tests.

Conformance tests

Every numeric rule requires safe and unsafe boundary cases, signed values, relevant overflow or precision cases, branches, overloaded/unknown variants and source-span tests. Compare exact modeled operations against isolated language executions in the test suite. Production static analysis still executes no repository code.

Add a fixture where Python `xs[-1]` is valid for a nonempty built-in list; a Kotlin negative array index is not. Add Number-versus-BigInt JavaScript cases before enabling that adapter.

Example lowering

```
def ratio(a: int, b: int):
    d = a - b
    if d != 0:
        return 100 / d
    return 0.0

domain: a,b are exact Python built-in integers (declared assumption)
entry:
  v0 = PARAM a
  v1 = PARAM b
  v2 = SUB v0 v1 [python.bigint]
  v3 = COMPARE NE v2 0
  BRANCH v3 nonzero zero
nonzero:
  v4 = DIV 100 v2 [python.true_division]
  RETURN v4
zero:
  RETURN 0.0
```

The division obligation in the nonzero block is `v2 != 0 AND v2 == 0`, which is unsatisfiable. This proves the zero-denominator failure absent for that block under the domain assumptions; it does not prove freedom from every other possible error.

05. Static engine

docs/05_STATIC_ENGINE.md

M1 supported subset

Support module-level named Python functions containing scalar parameters, exact built-in integer literals, assignments, integer addition/subtraction, simple multiplication where sound, comparisons, if/else, early return and division operators. Domains may be explicitly declared or inferred from exact literals. Annotated domains remain conditional.

Reject or mark relevant coverage unknown for decorators, arbitrary objects, overloaded operations, unknown calls, exceptions, loops, comprehensions, generators, async, reflection and dynamic imports. Do not parse only the easy expressions and present the whole function as verified.

M2 can add fixed built-in lists/tuples, modeled lengths/indexes and narrow None-flow checks with exact receiver types. Further support is explicit capability expansion.

Analysis order

1. Validate syntax and supported semantic domain.
2. Build CFG and symbol/value mapping.
3. Propagate constants and sound intervals to a fixpoint for supported acyclic flow.
4. Refine branch facts and nonzero/None states.
5. Collect obligations at risky operations.
6. Resolve cheap definite cases before constructing solver queries.
7. Encode supported remaining queries with a per-query time limit.
8. Interpret solver SAT/UNSAT/UNKNOWN using precision and coverage rules.
9. Emit source-mapped results and unsupported obligations.

Intervals need top, bottom, finite/infinite bounds, join, intersection and arithmetic transfer. Track nonzero constraints separately because a single interval cannot express a gap around zero. A reachable join unions facts; it must not preserve facts true on only one branch.

Division

For each supported `/`, `//` or numeric `%`, ask whether the divisor can be zero on a feasible path. Focus M1 on zero-denominator failures, not full arithmetic equivalence.

- Divisor interval excludes zero: safe for this obligation under domain assumptions.
- Constant zero on a reachable exact path: unsafe.
- Interval includes zero: uncertain; do not declare unsafe solely from the interval.
- Affine path over declared exact integers: optional SMT query.
- Unknown overloaded operation: unknown semantics.

Encode supported integer addition/subtraction and comparisons with mathematical integers only for the Python bigint profile. For fixed-width Kotlin integers later, use bitvectors and the correct signed-operation semantics.

An `assert` is not automatically a trusted runtime guard because Python can disable assertions. Record runtime mode or use explicit guarded control flow in proof fixtures.

Bounds

For built-in Python sequence length `n`, an index `i` is valid when `-n <= i < n`. Empty sequences admit no integer index. Do not apply this condition to slices, dictionaries, NumPy objects or user-defined `__getitem__`.

Model only known built-in sequence semantics and length facts. An unknown call that can mutate a list invalidates its length summary. Bounds through aliases are unknown until alias/effect support is sound.

None flows

Track None, definitely non-None and unknown as a small lattice. Attribute access on None can fail, but exact exception behavior of arbitrary receivers remains unknown. Treat `if x is not None` as a refinement. Mutating callbacks can invalidate object facts.

Do not conflate Kotlin nullable types, TypeScript undefined and Python None into one universal dereference rule.

Interprocedural summaries

M2 begins with direct named calls within a resolved local module. A summary describes declared/required domain, possible return values, modeled failures and effects. A proof at a caller is conditional on the summary preconditions; verify those preconditions.

Calls through unresolved names or dynamic references widen the result and effects. Recursive strongly connected components initially use conservative unknown summaries; record full coverage gaps. Later add widening and fixed-point summaries with explicit bounds.

Solver boundary

Use an optional Z3 adapter that builds typed expressions, stores query digests and translates supported concrete witnesses to JSON. Never allow model-provided SMT text into the solver unchecked. Enforce solver timeout and cancellation; track SAT, UNSAT and UNKNOWN separately.

Default design budgets: about 100 ms per simple query and 2 seconds for the quick static run, configurable and subject to measurement. Exceeding a required budget produces incomplete coverage. These are proposed defaults, not performance claims.

For exact modeled SAT, check the witness against the typed IR evaluator as an additional consistency check. The evaluator is not proof of frontend correctness; semantic conformance tests validate that boundary. For approximate SAT, runtime replay can establish an observed failure if enabled, otherwise keep POSSIBLY_UNSAFE.

Differential attribution

Compare obligations in base and candidate snapshots. A diagnostic identity alone does not determine introduction: a changed denominator or caller precondition can expand a pre-existing failure domain. Prefer replaying the same static witness in both modeled versions. If matching is ambiguous, relation is UNDETERMINED.

Report both function-domain failures and reachable-call evidence. A helper admitting all integers may have a failure that a verified caller excludes. Do not claim repository-level reachability without proving or observing it.

Required adversarial fixtures

Guarded divisor; unreachable bad branch; two-variable equality; unknown external call; Python bool/subclass domain issue; negative list index; empty list; alias mutation; widened caller domain; exception handler; syntax error; solver timeout; solver unavailable; unsupported float/NaN semantics.

A solver is an escalation mechanism, not a reason to encode every operation as unconstrained mathematical arithmetic.

06. Contracts and specification discovery

docs/06_CONTRACT_ENGINE.md

What a contract is

A contract is a typed predicate with a scope, precondition/input domain, authority state and evidence history. It can describe return ranges, length relations, selected purity/mutation behavior, serialization round trips, repeatability or other supported properties.

Repository-specific contracts are central to the project. Their intent must not be inferred solely from the fact that old code happens to satisfy them.

Candidate sources

Read explicit configuration, assertions, tests, docstrings, types, direct call sites and local model proposals. Runtime traces are optional evidence produced by explicitly configured execution; never inject hidden instrumentation into a user's application.

Preserve which source suggested which claim. Tests/docs can disagree. A test is evidence about tested cases; a documented contract becomes DECLARED only when an explicit adapter/configuration maps it to the typed predicate. Do not turn natural-language guesses into an approved specification automatically.

Lifecycle

```
CANDIDATE -> VALIDATED -> CHECKED -> REVIEWED
                |           |
             rejected    counterexample / unknown / evidence
```

Authority evolves independently: INFERRED -> APPROVED or REJECTED; a maintainer may also DECLARE a contract directly. Approved changes produce a new revision; obsolete contracts become RETIRED. A proof can establish that a candidate holds within a domain without establishing that it is desired behavior.

Minimal typed property DSL

Represent expressions as typed JSON nodes, not executable strings. v1 nodes:

- references: `arg(name)`, `result`, `before_arg(name)`, `after_arg(name)`;
- scalar operations: `literal`, `eq`, `lt`, `le`, `and`, `or`, `not`;
- controlled built-in operations: `len`, `sum`, `all_finite`, `approx_eq`;
- bounded collection predicates: `all_between`;
- relation templates: `unchanged_argument`, `repeatability`, later `round_trip`.

Each node has a declared accepted type and operation budget. `approx_eq` requires absolute/relative tolerance and explicit NaN/Infinity handling. A contract involving mutable arguments must specify copy strategy and supported serialization. Cycles, custom objects and equality methods remain unknown unless a trusted adapter exists.

Do not let a predicate call arbitrary repository code. A future round-trip template uses named allowlisted targets managed by the runner, not arbitrary expression execution.

Example contract

```
{
  "schema_version": "1.0",
  "contract_id": "normalize.sum.v1",
  "scope": "demo/math_ops.py::normalize",
  "authority": "APPROVED",
  "precondition": {
    "kind": "and",
    "args": [
      {"kind": "gt", "left": {"kind": "len", "value": {"kind": "arg", "name": "xs"}}, "right": {"kind":
    "literal", "value": 0}},
      {"kind": "all_between", "value": {"kind": "arg", "name": "xs"}, "lower": 0.0, "upper": 100.0},
      {"kind": "gt", "left": {"kind": "sum", "value": {"kind": "arg", "name": "xs"}}, "right": {"kind":
    "literal", "value": 0.0}}
    ]
  },
  "predicate": {
    "kind": "approx_eq",
    "left": {"kind": "sum", "value": {"kind": "result"}},
    "right": {"kind": "literal", "value": 1.0},
    "atol": 1e-9,
    "rtol": 1e-9,
    "require_finite": true
  },
  "input_profile": "bounded_builtin_numeric_list_v1",
  "provenance": [{"kind": "maintainer_approval", "reference": "contract-review-1"}]
}
```

Implement `gt` as a strict comparison node alongside `lt/le`, or canonicalize it during validation. Only these declared operations are accepted. The input profile also restricts lists to nonempty finite built-in numbers; JSON alone is not a complete generator.

Candidate validation

Verify schema version, known symbol, types, allowed operations, resource limits, noncontradictory domain and provenance. Check that the precondition has at least one admissible witness. Reject vacuous properties and missing domains. Treat an unsatisfiable precondition as INVALID_CONTRACT, not a proof of success.

Static checking uses only operations the analyzer faithfully models. Dynamic checks use the exact approved predicate evaluator and input profile. Unsupported predicate nodes yield unknown rather than best-effort evaluation.

Discovery and falsification

Start with deterministic templates: ranges, nonempty/length, explicit assertions and allowlisted purity contracts. The local model adds semantic candidates from bounded context packets. It must emit structured nodes, preconditions, source references and rationale.

Attempt falsification using boundaries, hand-authored inputs, existing examples and seeded property-based generators. Minimize a found counterexample within the valid domain. Never erase relevant preconditions during shrinking.

Keep discovery and evaluation sets separate. More observations can improve evidence coverage, but cannot justify universal numeric confidence or automatically approve intent.

Baseline and regression checks

For every candidate witness:

1. Validate the input against the same contract domain.
2. Run the baseline in its pinned environment.
3. Run the candidate in the corresponding environment.
4. Evaluate the same contract revision against both.
5. Classify baseline pass + candidate fail as regression.
6. Classify both fail as pre-existing or invalid specification.
7. Classify missing/incompatible baseline as UNDETERMINED.

A normalization failure on all-zero input is not a regression if the baseline also divides by zero. Use a positive-sum input in the demo.

Contract changes and stale evidence

Changing predicate, tolerance, domain, target symbol or library model invalidates the previous evidence for the new revision. Keep the old evidence as history. Deleted or renamed targets require explicit mapping or retirement. A model may suggest a revision but cannot approve it.

Determinism and mutation

Repeatability contracts define seed, number of repeats, environment controls and output comparison. Passing repeats provide TESTED evidence. A patch failure should be repeated to distinguish observed instability from a runner problem.

Mutation contracts use supported deep snapshots of arguments before/after; unknown custom serializers or object identity produce UNKNOWN. A property requiring immutability must be approved or declared before a mutation is blocking.

07. Incremental analysis and baselines

docs/07_INCREMENTAL_AND_BASELINES.md

Snapshot contract

A snapshot is a content-addressed source manifest with repository identity, file paths/digests, relevant configuration, analysis profiles and dependency lock digests. Results always point to two immutable identities: base and candidate. A commit hash alone does not capture uncommitted candidate changes.

Read the file bytes once and analyze those copies. Before finalizing, confirm source manifest consistency; if files changed mid-capture, restart once or return INCOMPLETE. Never combine symbols read from different working-tree moments.

Git and non-Git operation

`guard check` accepts a directory without Git. Patch verification requires an explicit saved baseline or Git reference. Never infer a base from an unstated merge base.

For Git-backed repos include tracked modifications, staged and unstaged changes, untracked supported files, deletions and renames. Snapshot lookup reads objects without changing the working tree. Use Git command argument arrays; never build shell strings from filenames or references.

A non-Git repository can use `guard baseline create --label NAME`. Store a source manifest plus explicit replay-source retention only when configured. Without retained source or an accessible revision, dynamic baseline replay may be unavailable and must say so.

Exclude .git, .guard, virtual environments, caches, vendor directories and configured paths. Avoid following symlinks outside the root. Report excluded changed files when they affect requested scope.

Graph model

Nodes: modules, symbols, globals, types, contracts and library models. Edges: calls, imports, references, writes, type use, contract dependencies and dynamic-resolution uncertainty. Store base and candidate graphs; removals can matter as much as additions.

Seed the affected slice with changed/deleted/new symbols, relevant module-level initializers, configuration/lock changes and approved contract revisions. Walk dependency edges in both versions. A changed callee can invalidate callers even when their text is unchanged.

Direct import graphs alone are insufficient. Effects on shared state and unresolved dynamic edges require conservative widening. Include affected strongly connected components for recursion. When exact slicing is impossible, reanalyze the whole containing module or configured repository scope.

Soundness versus speed

A smaller slice is useful only if dependencies are not missed. The first implementation should prefer too much reanalysis over unsound reuse. Compare incremental obligations with a full run on semantic fixtures as a release test. If a widening exceeds the quick budget, report INCOMPLETE and offer a full mode.

Global configuration, environment locks, compiler flags, modelled library versions or semantics-profile changes can invalidate all summaries. Do not reuse an old proof because the function body did not change.

Cache key

Use a digest of normalized relevant IR, raw source snapshot references, transitive summary dependencies, contract revision, semantic profile, analyzer/solver version, relevant configuration and environment identity. Analysis modes and bounds are part of the key.

Separate parse, graph, summary, solver and probe caches. A successful parse is not a passing verification cache entry. Reuse only completed evidence whose assumptions and coverage remain valid. Interrupted or failed entries do not count.

Do not put source line numbers in semantic identities. Remap current spans after reuse while keeping historical evidence spans tied to the original snapshot.

Matching before and after

Match symbols by qualified name/module plus signatures and conservative similarity for renames. Match obligations by property kind, operation and structural anchor. If a match is uncertain, keep both findings and mark the relation UNDETERMINED.

For a suspected introduced failure, evaluate the candidate witness in the base model as well as the candidate model. For execution evidence, replay it in both snapshots. Report newly unsafe caller paths even when a helper already had a potential failure.

Baseline lifecycle

A baseline is an explicit immutable revision, not the last scan result. A passing scan cannot automatically replace it. Creating or updating a baseline records the chosen revision, reason and contract set. The agent integration should have read/check access by default; changing approved baselines/contracts is a separate explicit command.

Keep baseline probes and learned candidates with their original domains. Do not train discovery on the candidate and call the candidate behavior a baseline.

Required change cases

Function body, signature, shared constant, library-version lock, module initializer, new file, removed file, rename, direct caller/callee, unresolved callback, recursion, contract revision, changed analyzer version and mid-run edits.

Fixture acceptance: changing a divisor returned by a callee must invalidate an unchanged caller. Changing a global limit must recheck a dependent index. Changing only a comment may reuse semantic analysis after span remapping, but docstring changes can affect candidate discovery.

08. Local open-weight reasoning

docs/08_LOCAL_MODELS.md

Product role

The model discovers semantic contract candidates and helps explain evidence. It never decides truth, approves intent, modifies a baseline, executes code or supplies trusted proof assumptions. Local inference is part of the integrated product, while the deterministic path remains available on machines without a capable model.

LocalReasoner protocol

```
capabilities() -> model identity, limits, structured-output support
propose_contracts(ContextPacket) -> CandidateBatch
propose_probe_parameters(Contract, InputProfile) -> BoundedInputCandidates
explain_finding(EvidencePacket) -> OptionalExplanation
```

The initial adapter targets a configured loopback inference endpoint. A second adapter can embed a runtime later. Do not equate an OpenAI-compatible API schema with cloud use: validate the destination independently.

No model access is needed for static checking, contract listing or existing evidence explanation through deterministic templates.

Context packet

Include only the affected function, directly relevant callers/callees, type facts, selected docstrings/tests, existing approved contracts and explicit unsupported areas. Set byte/token budgets. Never send the entire repository automatically.

Packet fields: schema_version, snapshot_id, symbol_ids, source_spans, source_digest, snippets, trusted_static_facts, candidate_domain, existing_contract_ids, unsupported_constructs and resource_budget.

Treat all snippets, comments and docs as untrusted data, including instructions that tell the model to ignore verification rules. The adapter has no tools for executing commands. Output passes strict validation regardless of prompt wording.

Candidate response

Each candidate must contain:

- a typed DSL precondition and predicate;
- scope and input profile;
- references to actual source spans;
- a concise rationale;
- explicit uncertainty and unsupported dependencies;
- optional boundary input proposals conforming to the allowed input schema.

Reject arbitrary Python, shell, SMT text, unsupported predicates, nonexistent symbols, invalid source references and excessive nested expressions. Candidate rationales are not evidence.

Probe parameters are values, not executable tests. The contract engine selects the approved template and generator; the runner executes it only within the configured boundary.

Model selection procedure

Evaluate at least two feasible open-weight options against deterministic template discovery and a no-model baseline. Record exact artifact ID/revision, license, quantization, runtime version, prompt schema, context limit, hardware, peak memory and time.

Use small hardware tiers: CPU/no-model, modest-memory local model, and larger-memory local model. Do not require every developer to load a large model. Keep installation/download an explicit step separate from verification.

Pin one tested recommendation only after this experiment. Model names, memory requirements and licensing from brainstorming are not product facts.

Quality measures

Candidate schema validity, nonvacuous-domain rate, mechanically checkable rate, useful accepted properties, counterexamples found, false intended-contract suggestions, tokens/time/memory and repeated-run variance. Compare with simple templates. Report developer review separately from mechanical proof.

Use held-out repositories and blind property review when possible. Do not use the same prompt-selected examples to claim general effectiveness.

Privacy and failure behavior

Local endpoint is loopback by default; reject redirects and non-loopback destinations. Support IPv4 and IPv6 validation and require explicit configuration for any future remote endpoint. No cloud fallback, telemetry or model download during verification.

If inference fails, continue deterministic work. Mark discovery unavailable and incomplete only if the requested discovery task requires it. Existing static obligations are not unknown solely because the model is offline.

Model output, prompts and explanations can contain code or secrets. Store raw context only when explicitly configured; default to a digest and minimal provenance. Findings and witnesses may also be sensitive, so apply the artifact retention policy.

Optional explanations

Prefer deterministic templates for source, path, domain, input and exception. A model explanation must cite existing evidence IDs and may not add a new proof claim. Validate references and preserve the structured report as authoritative. If explanation contradicts the evidence, discard it.

09. Interfaces and storage

docs/09_INTERFACES_AND_STORAGE.md

Intended CLI

These commands are proposed interfaces to implement milestone by milestone, not an existing installation.

Command	Behavior
guard init	Write minimal local configuration and ignore suggestions; run nothing
guard capabilities	Report installed adapters, solver, model and isolation capabilities
guard check PATH --format json	Static single-snapshot analysis
guard baseline create --label NAME	Save explicit immutable baseline metadata
guard verify --base REF --mode quick --format json	Compare base with captured working tree
guard verify --base REF --mode full	Broaden static scope with separate budget
guard verify --base REF --execute --run-profile NAME	Add explicitly configured dynamic contract checks
guard contracts discover --model local	Propose and mechanically check candidates; do not approve
guard contracts list	Show current contracts, evidence and authority
guard contracts approve ID --reason TEXT	Record explicit authority change
guard contracts revise ID --file PATH --reason TEXT	Create contract revision; invalidate stale evidence
guard contracts reject ID --reason TEXT	Retain history while excluding candidate from policy
guard explain FINDING_ID	Read existing evidence and explain it
guard replay FINDING_ID --run-profile NAME	Reproduce stored witness through authorized runner
guard mcp serve --stdio	Serve check/read tools over stdio

Baseline creation must specify source identity and whether replay-source retention is enabled. Static baseline metadata alone does not imply dynamic replay availability. `--mode full` never enables execution implicitly.

`guard check` has no patch attribution; its relation is UNDETERMINED. `guard verify` requires an explicit Git reference or saved baseline.

Exit and output contract

Exit	Meaning
0	PASS or WARN, with complete required coverage
1	BLOCK; report may also contain incomplete coverage
2	INCOMPLETE with no higher-priority block/error
3	ERROR in verifier/tool environment
4	Invalid CLI usage or invalid configuration

Return structured JSON on stdout when requested. Human diagnostics and progress go to stderr. Do not mix logs, model tokens or stack traces into JSON. Make output ordering deterministic apart from explicit run IDs/timing fields.

Report v1 example

This is illustrative, with symbolic IDs and example durations. Implementation must generate these values from real analysis.


```
{
  "schema_version": "1.0",
  "run_id": "run-example",
  "base_snapshot": "sha256:base-example",
  "candidate_snapshot": "sha256:candidate-example",
  "run_outcome": "COMPLETE",
  "verdict": "BLOCK",
  "policy_id": "default-v1",
  "coverage": {
    "requested_symbols": 2,
    "analyzed_symbols": 2,
    "required_obligations": 2,
    "resolved_required_obligations": 2,
    "unknown_obligations": [],
    "skipped_checks": []
  },
  "timing_ms": {"capture": 0, "static": 0, "dynamic": 0, "model": 0},
  "findings": [
    {
      "finding_id": "finding-example",
      "symbol_id": "demo/math_ops.py::ratio",
      "property_id": "division.zero_denominator",
      "property_result": "PROVED_UNSAFE",
      "scope_validity": "EXACT_MODEL",
      "authority": "DECLARED",
      "patch_relation": "INTRODUCED",
      "severity": "ERROR",
      "source_span": {
        "path": "demo/math_ops.py",
        "start_line": 3,
        "start_column": 11,
        "end_line": 3,
        "end_column": 22,
        "snapshot_id": "sha256:candidate-example"
      },
      "quantifier": "exists admitted function input",
      "domain": "exact Python built-in integers",
      "assumptions": ["domain explicitly admitted in configuration"],
      "path_constraints": ["a == b", "d == a - b", "d == 0"],
```

```
      "witness": {"a": 1, "b": 1},
      "evidence_ids": ["evidence-example"],
      "replay_available": false
    }
  ],
  "obligation_results": [],
  "capabilities": {"solver": "configured", "execution": "disabled", "model": "disabled"}
}
```

Production schemas must reject placeholder IDs in stored results. The example shows a function-domain failure, not a claim that a particular production caller reaches it. Store successful obligations as well as findings.

MVP schema checklist

Create machine-validated schemas for Report, Finding, ObligationResult, Evidence, SourceSpan, Snapshot, ContractRevision, CandidateBatch, ContextPacket, RunProfile and AnalysisPlan. This kit defines the required fields and enums; M0 creates the actual JSON Schema files and validation tests.

Allow optional forward fields only through an explicit schema-version policy. Unknown enum values are not silently interpreted as safe. Fields documenting scope/domain and coverage remain required.

Source locations use one-based lines and zero-based UTF-8 byte columns, with exclusive end positions. Python AST byte offsets must be normalized carefully. Kotlin adapters must convert their native offset conventions to the same schema.

Configuration draft

```
schema_version = "1.0"

[analysis]
languages = ["python"]
mode = "quick"
solver_enabled = true
per_query_timeout_ms = 100
quick_budget_ms = 2000
require_coverage = true

[execution]
enabled = false
profile = ""
allow_unisolated = false

[semantic]
enabled = false
provider = "loopback"
endpoint = "http://127.0.0.1:PORT"
model_id = ""
download_during_scan = false

[privacy]
telemetry = false
store_raw_model_context = false
retain_replay_source = false
store_witnesses = true

[scope]
exclude = [".git", ".guard", ".venv", "vendor", "node_modules"]

[[domains]]
symbol = "demo/math_ops.py::ratio"
profile = "python_exact_builtin_ints"
authority = "DECLARED"
reason = "demo public API admits exact built-in integer inputs"
```

The endpoint is an unset illustrative placeholder; `guard init` should omit it or leave an empty value until a real endpoint is configured. Never attempt to connect to this example literally.

MCP integration

Expose `verify_patch`, `check_scope`, `get_contracts`, `get_finding`, `explain_finding` and `get_capabilities`. They call the same orchestration code as the CLI and return versioned structured reports.

`verify_patch` inputs: repository alias/root restricted to configured roots, base reference, mode and optional preauthorized execution profile. Return report identity plus compact findings and coverage; allow fetching detailed evidence by ID.

Do not expose baseline replacement, contract approval or arbitrary command execution as default agent tools. Cancellation, timeouts and stale snapshot errors have explicit responses. Use stdio for local integration; stdout is reserved for protocol messages.

An agent convention can request verification before completion. MCP alone cannot force every agent to comply. Enforced acceptance requires client/hook/CI integration, which must be tested separately.

Persistent layout

```
.guard/
  config.toml
  state.sqlite
  contracts/       # approved declarations/revisions in reviewable JSON
  artifacts/       # content-addressed evidence and optional replay data
  snapshots/       # manifests; source only if retention enabled
  cache/
  runs/
```

SQLite owns identity, relationships, indexes and run transactions. Artifacts are immutable by digest. Contracts retain readable JSON revisions; avoid making SQLite and files competing editable sources of truth. Use explicit import/export and atomic revision transactions.

Recommended ignore rules exclude cache, local run output, sensitive artifacts and retained source. Whether approved contracts/configuration belong in Git is a maintainer choice; never commit these automatically. Implement schema migrations before writing a second schema version.

10. Execution boundary and privacy

docs/10_EXECUTION_AND_PRIVACY.md

Static mode is the default

Parsing, graph construction, abstract interpretation and solver queries operate on source bytes and trusted analyzer code. They must not import repository modules, run package installation, invoke build scripts, execute Git hooks, or evaluate model-generated expressions.

Compiler-backed adapters require an explicit capability boundary because compiler plugins or build systems can execute code. A Kotlin sidecar should compile configured isolated sources with a trusted version-pinned compiler rather than automatically run arbitrary Gradle builds.

Dynamic operation contract

A dynamic command requires both an explicit `--execute` request and a named run profile enabled in local configuration. MCP can use only preauthorized profiles; it cannot make a new permission decision or create one.

A profile specifies:

- allowed target modules/functions and typed input templates;
- pinned interpreter/compiler and environment lock digest;
- isolation backend and verified capabilities;
- network policy, writable paths, CPU/memory/time/process limits;
- working-directory mounts and environment-variable allowlist;
- maximum input/output/artifact sizes;
- baseline/candidate environment mapping;
- source-retention and witness-storage settings.

Do not run a target import until the isolation boundary has been established. Importing Python code is already execution.

Isolation requirement

Preferred runner: a verified container/OS isolation backend with read-only source copies, temporary writable scratch, denied network and no host credentials. A container by itself does not automatically provide every guarantee; probe and report each capability.

If the backend cannot enforce the required profile, return RUNNER_UNAVAILABLE or INCOMPLETE. A timeout-killed subprocess provides resource control but not filesystem/network isolation.

An explicit trusted-repository profile may enable unisolated execution only through `allow_unisolated=true` plus the execution command. Label the evidence with actual capabilities. Never call this execution sandboxed.

The product should not add repeated prompts during an already authorized configured workflow. The configuration is the explicit reusable execution authorization.

Probe implementation

A trusted runner imports a copied snapshot in the configured environment, invokes allowlisted function targets with typed serialized inputs, captures supported results/exceptions, evaluates predicates and returns structured observations.

Model-produced text cannot select shell commands, imports, environment secrets or mount paths. Validate generated values against the input schema and resource limits.

Use argument arrays for process invocation. Reject path traversal, symlink escapes, unauthorized targets and oversized/cyclic values. Kill the process group on timeout where supported; confirm termination before reusing the environment.

Baseline comparability

Both sides need suitable pinned dependencies and semantically comparable run profiles. Do not automatically install packages while replaying. Prepare environments through an explicit setup workflow.

If the patch changes dependencies, record both lock files and assess whether a witnessed difference is a intended dependency/API change. Do not silently force the baseline to run with the patch environment. An unavailable historical dependency is UNDETERMINED comparison.

Data handling

No cloud inference or telemetry by default. Local model requests use only bounded necessary context. Reject non-loopback endpoints and redirects in the local adapter. Verification should work offline after installation.

Reports, exception text, witnesses, source snippets and contract provenance can contain sensitive data. Limit captured output, exclude environment secrets, and support redaction plus configurable witness/source retention. A redacted witness may become nonreplayable; state that honestly.

Raw model context is off by default. Store hashes and minimal source references. Source retention for baseline replay is also off by default; Git revisions may provide baseline source without a second retained copy.

Repository integrity

Never modify a checked snapshot or working tree. Runner writes only to temporary locations. Do not reset Git, stash user changes, edit dependencies or install hooks without an explicit setup command. Concurrent user edits result in snapshot restart or incomplete comparison.

`guard init` writes only its declared configuration/metadata and prints suggested ignore rules. CI/hook installation is a separate operation.

Operational acceptance

Test that static scans of a module with a top-level file write do not execute it. Test that dynamic isolation denies outbound network, host writes and credential access under its stated profile. Test cancellation, output limits, malicious candidate nodes and malicious filenames. If platform isolation is unavailable, tests must confirm fail-closed behavior.

This boundary supports the product's local/private claim. It is not a claim to solve arbitrary malicious-program containment on every operating system.

11. Roadmap and release gates

docs/11_ROADMAP_AND_GATES.md

Build order and estimates

The order below is dependency-driven. Effort ranges are planning estimates for one experienced builder with focused implementation time; they exclude unknown research-code integration and are not calendar promises. Milestone gates matter more than dates.

Milestone	Scope	Estimated focused effort	Depends on
M0	Package, schemas, fixtures and status record	2-4 days	Nothing
M1	Python static vertical slice	1-2 weeks	M0
M2	Impact, baselines, caches and added checks	1-2 weeks	M1
M3	Contracts, isolation and differential probes	2-3 weeks	M2
M4	Real local model discovery	1-2 weeks	M3 typed DSL
M5	MCP and measured agent repair demo	1 week	M2-M4
M6	Kotlin/JVM adapter beta	2-4 weeks	M1 IR; M5 integration
M7	TS/library adapters and broader support	Open-ended	Validated earlier releases

Work can be interleaved when interfaces are stable, but no gate should be declared complete using unimplemented capabilities. This plan does not require parallel agents.

M0: foundation

Deliver source package, test setup, schema validation, provisional command entrypoint, fixture manifests, first decision record and IMPLEMENTATION_STATUS.md. Schemas include assumptions, coverage and authority from the beginning.

Gate: package installs in a clean local environment; schema invalid/unknown cases fail validation; tests run without model/network; source maps and snapshot identity are represented.

M1: single-snapshot static loop

Implement AST-to-IR lowering, integer domains, CFG, intervals, nonzero/path facts, division obligations, optional SMT and structured CLI. Add supported/unsupported coverage reporting.

Gate: the safe/unsafe/unknown fixtures produce actual semantic outcomes; no repository import occurs; solver unknown and engine error are distinct; a conditional domain never becomes unconditional through annotations.

Demonstrate a concrete witness for unsafe ratio and a guarded safe obligation. Do not claim patch attribution before M2.

M2: patch verification

Add explicit snapshots, Git/non-Git baselines, diff-to-symbol mapping, dependency graph, conservative slices, summary invalidation, before/after static comparison, basic list bounds and narrow None checks.

Gate: unchanged callers are rechecked when a callee changes; new/untracked/deleted files participate; incremental and full required obligations agree on fixtures; stale caches fail closed; no checkout or working-tree mutation occurs.

Measure static timing and memory. Capture unsupported dynamic edges rather than hiding them.

M3: behavioral contracts

Implement typed predicate/input DSL, lifecycle/authority, immutable evidence, deterministic template discovery, isolated runner, baseline replay, shrinking and approved-contract policy.

Gate: normalize regression demonstrates baseline pass/candidate fail on the same positive-sum input; invalid-domain and both-fail cases are not regressions; passing probes say TESTED; runner unavailable means incomplete when dynamic checks are required.

At this gate, the core dual-engine architecture exists. Contracts may be declared or template-mined; model inference is still pending.

M4: open-weight semantic layer

Implement local adapter, bounded context, structured candidate validation, candidate falsification and hardware/model evaluation. Configure one actual open-weight artifact; record its tested license and runtime identity.

Gate: real model-generated candidates enter the same mechanical pipeline as deterministic candidates; invalid/vacuous/adversarial suggestions are rejected; no cloud/network fallback occurs; model-off remains functional.

This gate is necessary for the full local/open-weight product thesis. A mock model response is useful for unit tests but cannot satisfy release acceptance.

M5: agent-native alpha

Implement stdio MCP, shared APIs, cancellation and compact evidence retrieval. Document one tested client integration and demonstrate patch -> verify -> failure evidence -> repair -> verify.

Gate: the report tracks the final snapshot; no stale evidence is attributed to the repaired code; unknown required work prevents PASS; the agent never auto-approves a baseline/contract change. Measure the complete loop separately from verifier latency.

A hook or CI integration can enforce acceptance where the client supports it. Report client conventions accurately; MCP alone is not enforcement.

M6: Kotlin beta

Run the compiler-adapter feasibility spike. Pin Kotlin/JVM/compiler settings. Implement typed lowering and integer/bounds/null-assertion checks, then reuse patch scope, evidence and interfaces.

Gate: Kotlin fixtures match compiled isolated executions; overflow and null semantics are not translated as Python arithmetic; adapter version changes invalidate summaries; no automatic arbitrary Gradle execution.

If research code is supplied later, evaluate reuse at this milestone. The plan does not imply its source, sponsor permission or dependencies are already available.

M7: expansion

TypeScript first where user demand supports it, including undefined propagation, Number/BigInt distinctions and Promise-related checks with documented semantics. Add versioned tensor-shape, DataFrame-schema and JSON-contract models after small controlled fixtures.

Only add a new language when its typed adapter and semantic fixtures pass. Java/Go/Rust/C/C++ are architecture directions, not simultaneous launch commitments.

Release criteria

Before an alpha: M0-M5 gates, working no-model path, actual local model path, supported-subset docs, schema-valid reports, replayable regression, offline operation and one real client repair integration.

Before Kotlin beta: M6 gate plus shared-schema compatibility and performance/coverage regression checks.

Before public package release: check naming, choose source license, audit shipped dependency/model licenses, document limitations and install commands, create a clean versioned release. No publishing is authorized merely by reading this kit.

Reduce scope if blocked

If SMT support is difficult, ship interval/path results with UNKNOWN on unresolved queries while continuing solver work. If isolation is missing, keep static verification useful but do not mark M3 dynamic acceptance complete. If model hardware is weak, use a smaller evaluated open-weight model or no-model mode; do not substitute an undisclosed cloud call. If Kotlin compiler internals are unstable, pin the adapter or postpone the gate transparently.

Do not reduce scope by dropping assumptions, coverage or provenance. Those are core product behavior.

12. Testing, benchmarks and research

docs/12_TESTING_AND_RESEARCH.md

Validation layers

Layer	What it establishes
Unit semantics	Correct transfer rules, lattices, DSL typing and policy
Language conformance	IR/model behavior agrees with isolated language fixtures
Static integration	Real source reaches source-mapped report without imports
Differential replay	Baseline/candidate attribution uses comparable observations
Incremental equivalence	Cached/sliced results preserve required full-run obligations
Agent integration	A real client consumes evidence and repairs a real patch
Privacy/isolation	Actual default operation matches local/execution claims
Research benchmark	Measured utility and latency on held-out tasks

Do not test only that implementation constants repeat themselves. Each core rule needs a meaningful counterexample or boundary case.

Fixture manifest

Every case contains case ID, language/version, baseline source, patch source, property/domain, expected result and scope, ground-truth method, environment lock, seed and origin/license. Include clean safe cases and unsupported cases as well as defects.

Initial Python suites: unguarded/guarded division, impossible branch, annotation-only domain, unknown external call, direct caller precondition, callee change, Python negative index, empty list, None flow, mutable alias, NaN/float unsupported behavior, ignored assertion runtime mode, syntax error and mid-run edit.

Dynamic suites: normalization positive-sum regression, all-zero excluded domain, both versions already fail, purity regression, controlled repeatability, shrinking preserves domain, missing baseline environment and isolation denial.

Kotlin suite: Int division, overflow-sensitive path, array index, explicit `!!`, safe null guard and unresolved Java platform type.

Prevent fabricated precision

Report TP/FP/FN only for cases with independently established labels. Maintain safe negatives. Count unknowns and abstentions instead of removing difficult cases from the denominator.

A static function-domain witness is not necessarily a reachable application defect. Ground truth should record this distinction. A model-inferred property can be mechanically valid but undesired; developer intent review is a separate label.

Benchmark tracks

1. Controlled semantic fixtures: exact modeled safety, failures and unknown behavior.
2. Seeded realistic patches: hand-written changes in small repository examples, with independent tests.
3. Real agent patches: versioned tasks and baseline repositories, with disclosed generation and evaluation environment.
4. Repository contract utility: blind review of inferred properties and found counterexamples.
5. Incremental performance: full/cold/warm runs under the same required coverage.

Synthetic defects prove engineering behavior. They do not alone establish effectiveness on natural coding-agent mistakes.

Metrics

Precision/recall by failure class and domain; conditional/reachable distinctions; abstention rate; required-obligation resolution; incremental/full mismatch count; p50/p95 wall time; peak memory; cache hit/invalidation counts; model schema/checkable/accepted-candidate rates; replay availability; counterexample minimization; repair success and attempts; false blocks on intentional changes.

Separate total verifier time into capture, parsing, impact, abstract analysis, solver, dynamic checks and model. Warm static time excludes asynchronous discovery only when clearly labeled. End-to-end latency includes what the user actually waits for.

Proposed performance experiment

Use a reference laptop with disclosed OS, CPU, RAM, interpreter and model hardware. Test repository scales with approximately 100, 1,000 and 10,000 supported functions and changes spanning one, several and many dependency nodes. Include one highly connected global change.

Use at least 30 repeated timings per condition where feasible, recording cold/warm definitions. Run full and incremental modes at the same coverage. A two-second p95 quick-static target is provisional; publish measured outcomes rather than rewriting the target as achieved.

Baselines and ablations

Compare:

- generic static analyzer adapter alone;
- InvariantGuard static full versus incremental;
- declared contracts versus deterministic inferred templates;
- local model candidates plus mechanical verification;
- static-only, dynamic-only and combined verification;
- full dependency invalidation versus weaker experimental slicing.

Use REDO and agentic property-based testing as conceptual prior work. A direct benchmark requires reproduced code, compatible tasks and documented settings; do not put imaginary performance numbers into a competitor table. Licensing and language differences matter.

Research questions

Does conservative change-aware verification preserve useful detection while reducing latency? Do mechanically checked local-model candidates add useful repository properties beyond templates? Does baseline differential evidence reduce false blocks? How often does combining static feasibility and dynamic falsification improve actionable findings? What are the costs of honest abstention?

These are hypotheses, not established novelty or effectiveness claims.

Reproducibility

Lock code/dependency revisions, semantic profiles, compiler/solver/model artifacts, prompts, generation parameters, input seeds, environment manifests and benchmark splits. Version contracts and baseline identities. Store raw outcomes where retention permits.

Use held-out repositories split by repository family to reduce leakage. Do not tune rules on test results and then report the same cases as held out. Bootstrap intervals by repository when clustering matters. Report sample counts and finite uncertainty; no unearned per-finding probability.

Initial research deliverables

A documented defect/contract fixture set, measured cold/warm report, model-candidate evaluation, incremental/full consistency results and a recorded agent repair demo. A paper comes after reproducible empirical evidence, not before a benchmark exists.

13. Demo and acceptance scenarios

docs/13_DEMO_AND_ACCEPTANCE.md

Demo promise

Show a real patch introducing one modeled runtime-failure path and one approved behavioral regression. Return concrete inputs to an agent; apply a real repair; verify the final captured snapshot. All numbers in the demo report must come from implementation, not this document.

Fixture A: ratio

Baseline:

```
def ratio(a: int, b: int):
    d = a - b
    if d == 0:
        return 0.0
    return 100 / d
```

Candidate:

```
def ratio(a: int, b: int):
    d = a - b
    return 100 / d
```

Domain: exact Python built-in integer arguments, explicitly declared as admitted by the demo API configuration. Annotations alone would provide only conditional evidence.

Expected witness: `a=1, b=1`. Baseline returns 0.0; candidate has a zero denominator. Static output proves an existential failure in the declared modeled function-input domain. If execution is enabled, replay additionally observes ZeroDivisionError. This does not require claiming a production caller reaches the function.

Safe repair: restore an explicit zero guard. Its division obligation is safe under the declared domain. Other runtime errors remain outside that property.

Fixture B: normalization

Baseline:

```
def normalize(xs):
    total = sum(xs)
    return [x / total for x in xs]
```

Candidate:

```
def normalize(xs):
    total = sum(xs) + 1.0
    return [x / total for x in xs]
```

Approved contract: a nonempty finite built-in numeric list with elements in [0,100] and positive total should yield finite results summing approximately to 1, with atol=rtol=1e-9.

Witness `xs=[1.0,1.0]`: baseline returns [0.5,0.5], summing to 1. Candidate returns approximately [1/3,1/3], summing to 2/3. Replay both in isolated environments. This is OBSERVED_REGRESSION, not static proof over arbitrary lists or floats.

All-zero input is outside this contract. Both versions can behave differently there; it must not be used to claim this positive-sum contract regressed.

Safe repair: restore the original total calculation, or implement a deliberate new API and revise the approved contract explicitly. Passing generated probes afterward remains TESTED evidence.

Fixture C: honest unknown

```
def imported_ratio(a, b):
    d = external_transform(a, b)
    return 100 / d
```

Unknown call output/effects make the division obligation UNKNOWN in the first supported subset. No fabricated witness or unconditional PASS. Required coverage leads to INCOMPLETE.

Fixture D: changed callee

Baseline callee `denominator(x)` returns `x + 1` in an admitted domain `x >= 0`; caller divides by it. Candidate callee returns `x`. Caller source is unchanged. Input x=0 makes only the candidate divisor zero. The graph must invalidate the caller.

Fixture E: mutation

Baseline `sorted_copy(xs)` returns `sorted(xs)`. Candidate calls `xs.sort()` and returns xs. With an approved unchanged-argument contract and input [2,1], baseline preserves the argument while candidate mutates it. Check using supported pre/post serialized snapshots. Merely naming the function sorted_copy is not sufficient authority.

Demonstration sequence

After M5 is implemented, use a disposable example repository with no sensitive source:

1. Create a baseline commit with safe ratio and normalize.
2. Initialize configuration; declare the ratio domain and approve the normalization contract.
3. Configure an isolated Python run profile. Prepare dependencies explicitly.
4. Save baseline identity; apply the candidate changes.
5. Call `guard verify --base BASE_REF --format json --execute --run-profile isolated-python`.
6. Confirm introduced static evidence and true baseline/candidate regression.
7. Feed the report to the coding agent through CLI or MCP.
8. Agent restores the guard and normalization behavior.
9. Verify the repaired snapshot; show resolved findings, tested contracts and any remaining unknowns.
10. Show a separate no-model static run and actual local-model candidate discovery.

BASE_REF is a real captured reference selected during the demo, not a literal Git name to assume exists.

Acceptance matrix

Scenario	Required behavior
Unguarded ratio, declared domain	Exact modeled witness; admitted-domain block
Annotation only, unconfirmed callers	Conditional finding and advisory policy
Restored guard	Safe zero-denominator obligation within scope
Positive-sum normalization patch	Same input baseline pass/candidate fail
All-zero normalization input	Excluded domain, not this contract's regression
Inferred unapproved contract	Advisory change, no automatic intent authority
Both versions violate property	Pre-existing or invalid spec, no introduced-regression label
Unknown external call	Unknown obligation and incomplete required coverage
Changed callee	Unchanged caller is rechecked
Required solver timeout	Incomplete, never safe
Model unavailable	Deterministic mode still works
Dynamic backend unavailable	No unisolated fallback
Baseline dependencies missing	Comparison unknown; static work can still finish
Mid-run edit	Restart consistent capture or incomplete outcome
Deliberate contract revision	New audited revision; old evidence retained

Demo integrity

Keep baseline, patches, repairs, environment manifests, contract revisions and actual generated reports. Do not hand-edit findings or timings for the presentation. A concise terminal recording plus the JSON evidence is enough; a dashboard is unnecessary.

The demo establishes the implemented subset and repair workflow. It does not establish universal runtime-error prevention, arbitrary language support or sub-second performance.

14. Decisions, risks and reading

docs/14_DECISIONS_AND_REFERENCES.md

Decision record

ID	Decision	Why
D01	Shared typed verification core with semantic language profiles	Reuse analyses without erasing operator behavior
D02	Python first, Kotlin second	Prove an agent loop, then integrate the compiler-research direction
D03	CLI before MCP; common orchestration API	Keep one source of truth for report/policy behavior
D04	Optional local open-weight adapter, mandatory actual M4 release test	Preserve accessibility while shipping the semantic layer
D05	Finite tests never become universal proof	Avoid invented confidence
D06	Authority separate from mechanical validity	Old behavior and inferred properties may be incidental
D07	Conservative impact before aggressive slicing	Cache mistakes can hide real failures
D08	Static default; dynamic checks explicitly configured	Local analysis should not silently execute a repository
D09	No full universal IR/optimizer before narrow IR works	Prevent infrastructure from consuming the first release
D10	Evidence and baseline source tied to immutable snapshots	Make attribution and replay reviewable

Changes to these decisions must record the trigger, alternatives, evidence and effect on release criteria. The project definition remains dual verification plus repository contracts, local reasoning, patch scope and agent integration.

Main risks

Risk	Consequence	Mitigation
Dynamic-language imprecision	Too many unknowns or spurious static alerts	Tight explicit subset, exact fixture conformance and replay where enabled
Incidental inferred properties	False blocks on valid changes	Separate authority; require approval for behavioral blocking
Missed dependency effects	Unsound cache reuse	Conservative widening; full/incremental comparison fixtures
Model proposal errors	Nonsensical or unsafe predicates	Typed DSL, nonvacuous domains and mechanical checking
Slow interprocedural work	Verification disrupts agent loop	Budgets, summaries, async discovery and explicit incomplete status
Weak hardware	Local model inaccessible	Model tiers and functional no-model mode
Compiler API churn	Kotlin adapter breaks	Pin compiler sidecar; test invalidation and compatibility
Baseline environment drift	Misattributed regressions	Pinned environments and explicit undetermined comparison
Broad launch scope	No usable end-to-end release	Milestone gates before new frontends

Prior-art position

Static analysis, dynamic invariant discovery, symbolic checking and agent-generated property tests are established. REDO overlaps execution-free runtime-error detection for coding-agent patches. Agentic property-based testing overlaps automatic property inference and counterexample search. DIG and Daikon overlap invariant discovery/checking.

The intended contribution is an integrated product and an empirical investigation: repository-specific contract provenance plus change-scoped safety/behavior checks, local semantic proposal, and agent-facing evidence. Do not claim first invention or absence of competitors. A fuller current landscape and licensing review remains a pre-publication task.

Verified primary-source reading

Links were checked for this handoff on 4 October 2026. Use version-pinned documentation during implementation; documentation links can change. These sources support background/semantics, not claimed performance of InvariantGuard.

1. Python expression semantics: https://docs.python.org/3/reference/expressions.html

Built-in arithmetic, division and subscription rules. Read before implementing numeric/bounds rules; custom operations need separate treatment.

2. Python AST API: https://docs.python.org/3/library/ast.html

Parser structures and source locations. AST is syntax input, not a complete semantic verifier.

3. Kotlin numbers: https://kotlinlang.org/docs/numbers.html

Numeric types, division and overflow considerations for the Kotlin adapter.

4. ECMAScript numeric semantics: https://tc39.es/ecma262/multipage/ecmascript-data-types-and-values.html

Number and BigInt operations have distinct division behavior. Use the target runtime/spec profile.

5. Z3 repository: https://github.com/Z3Prover/z3

Solver integration, licensing and build/version reference.

6. Tree-sitter: https://tree-sitter.github.io/tree-sitter/

Incremental parsing for future indexing; parsing alone supplies neither type facts nor proof.

7. Hypothesis documentation: https://hypothesis.readthedocs.io/en/latest/

Generative testing and counterexample search. Passing cases remain finite evidence.

8. MCP transport specification: https://modelcontextprotocol.io/specification/2025-06-18/basic/transports

A versioned stdio/transport reference. Negotiate and pin the implemented protocol version.

9. Official MCP Python SDK: https://github.com/modelcontextprotocol/python-sdk

Candidate implementation dependency; pin and test a selected version.

10. REDO paper: https://arxiv.org/abs/2410.09117

Execution-free runtime-error detection combining LLMs and static analysis for coding agents; direct relevant prior work.

11. Anthropic property-based testing research: https://www.anthropic.com/research/property-based-testing

Agent-based property discovery, testing and bug finding; overlaps the dynamic branch.

12. DIG: https://github.com/dynaroars/dig

Numerical invariant generation from source/traces with checking machinery; relevant inference background.

13. Daikon: https://plse.cs.washington.edu/daikon/

Dynamic invariant detection; observed invariants require care about domains and intended specifications.

Open decisions for implementation

Choose exact dependency versions in M0; record the isolation backend by platform in M3; select tested local model artifacts in M4; choose compiler integration/version in M6. Source license, public branding, package naming and public deployment remain explicit release decisions.

Do not invent sponsorship rights, access to unpublished research code, compiler compatibility or model memory requirements. This kit gives a plan to evaluate those questions.

15. Ordered implementation backlog

docs/15_TASK_BACKLOG.md

Use this file as the implementation checklist. Check an item only after its completion evidence exists in IMPLEMENTATION_STATUS.md. Dependencies follow the milestone sequence. M1 is the first Claude assignment.

M0: foundation

[ ] T00 Create pyproject, src layout, provisional guard entrypoint and local test setup. Done: clean local install and CLI help.

[ ] T01 Create versioned report/evidence/contract/snapshot/IR schema models. Done: valid and invalid payload tests; assumptions/coverage required.

[ ] T02 Implement source spans and content digests. Done: Unicode-column and exclusive-end cases correct.

[ ] T03 Create fixture manifests and IMPLEMENTATION_STATUS.md. Done: each fixture identifies domain, expected evidence and ground truth.

[ ] T04 Create decision/update convention and dependency lock. Done: tested host version and selected dependencies recorded.

M1: Python static slice

[ ] T10 Parse real Python AST without imports. Done: top-level side-effect sentinel remains untouched.

[ ] T11 Resolve supported local symbols and lower typed minimal CFG/IR. Done: branch/source mappings validated.

[ ] T12 Implement integer constant/interval/nonzero states. Done: joins, bottom and safe/unsafe boundary tests.

[ ] T13 Implement division obligations and domain assumption tracking. Done: ratio safe/unsafe/annotation-only cases distinct.

[ ] T14 Add optional typed Z3 adapter, budgets and exact witness validation. Done: SAT/UNSAT/UNKNOWN/unavailable cases accurate.

[ ] T15 Implement unsupported effects/coverage. Done: external-transform case is UNKNOWN and required incomplete.

[ ] T16 Implement guard check JSON/human output and exit policy. Done: stdout schema valid, stderr logs separate.

[ ] T17 Pass M1 acceptance. Done: real source -> real findings; no mocked solver or hard-coded demonstration outputs.

M2: change-aware verification

[ ] T20 Capture immutable base/candidate snapshots, Git and non-Git. Done: staged/unstaged/untracked/deleted paths included.

[ ] T21 Map diffs to symbols and conservative before/after graph. Done: renames/deletions unresolved cases disclosed.

[ ] T22 Implement direct call summaries and reverse invalidation. Done: changed-callee fixture catches unchanged caller.

[ ] T23 Track global/type/config/lock/contract dependencies. Done: dependent summary invalidates.

[ ] T24 Implement cache validity and interruption handling. Done: stale or partial evidence never reused as pass.

[ ] T25 Implement list-bounds and narrow None-flow checks. Done: negative Python indexes, empty list and unknown aliases tested.

[ ] T26 Classify introduced/pre-existing/resolved/changed findings. Done: same witness checked against both models.

[ ] T27 Implement verify quick/full, coverage and baseline lifecycle. Done: no Git checkout or silent baseline update.

[ ] T28 Compare incremental/full fixtures and measure cold/warm. Done: no required-obligation mismatch; actual timing recorded.

M3: behavioral contracts

[ ] T30 Implement typed DSL validation/evaluator and bounded input profiles. Done: invalid, vacuous and malicious expressions rejected.

[ ] T31 Implement contract lifecycle, authority and immutable revisions. Done: approval separate from tested/proved result.

[ ] T32 Add deterministic candidate templates and provenance. Done: sources and assumptions retained.

[ ] T33 Implement run profiles and capability-checked isolation. Done: disabled/static default and fail-closed backend behavior.

[ ] T34 Implement isolated allowlisted import/invocation. Done: resource/network/host-write tests match claimed capabilities.

[ ] T35 Add differential replay and shrinking. Done: positive-sum normalize witness passes base and fails patch.

[ ] T36 Add supported mutation/repeatability templates. Done: observations remain finite; domain-preserving shrink.

[ ] T37 Implement evidence store and contract gate policy. Done: inferred change warns; approved replayed regression blocks.

[ ] T38 Validate stale evidence and intentional revision path. Done: new predicate/domain cannot inherit old passing evidence.

M4: local semantic discovery

[ ] T40 Implement ContextPacket and LocalReasoner protocol. Done: bounded source selection and candidate schema.

[ ] T41 Implement validated loopback adapter. Done: redirect/non-loopback destinations rejected; no cloud fallback.

[ ] T42 Integrate one real open-weight artifact. Done: revision/license/runtime/hardware recorded and actual inference demonstrated.

[ ] T43 Validate/refute candidate properties mechanically. Done: model output cannot produce proof/approval on its own.

[ ] T44 Evaluate templates, no-model and local-model candidates. Done: checkability/usefulness/time measured on held-out examples.

[ ] T45 Test offline/failure/context-retention behavior. Done: deterministic path survives model absence.

M5: agent integration

[ ] T50 Implement stdio MCP over shared orchestration API. Done: protocol output/log separation and schema compatibility.

[ ] T51 Restrict repository roots, tools and execution profiles. Done: default tools cannot approve contracts/baselines.

[ ] T52 Implement cancellation and evidence fetching. Done: cancelled/stale runs cannot become passing cache entries.

[ ] T53 Configure one real agent client. Done: exact tested integration instructions.

[ ] T54 Record complete repair loop. Done: actual unsafe patch, generated evidence, real repair and final snapshot report.

[ ] T55 Assemble alpha release checklist. Done: M0-M5 gates, accurate limits, offline and model/no-model paths.

M6: Kotlin

[ ] T60 Spike typed compiler integration. Done: written decision on pinned Kotlin/JVM adapter boundary.

[ ] T61 Implement versioned sidecar/IR exchange. Done: unsupported/version mismatch explicit.

[ ] T62 Add integer-division, array-bounds and null-assertion rules. Done: semantics match isolated compiled fixtures.

[ ] T63 Reuse impact/report/policy interfaces. Done: no parallel incompatible finding format.

[ ] T64 Validate overflow, platform types and compiler settings. Done: assumptions and invalidation correct.

[ ] T65 Pass beta gates. Done: shared core works across Python and Kotlin with disclosed coverage.

M7: expansion and evaluation

[ ] T70 Prioritize TypeScript against user demand. Done: documented adapter subset and Number/BigInt/undefined tests.

[ ] T71 Add selected versioned tensor/DataFrame/schema models. Done: real trusted library summaries and boundary fixtures.

[ ] T72 Build labeled held-out patch benchmark. Done: safe negatives, independent ground truth and licensed inputs.

[ ] T73 Run ablations and publish measured internal report. Done: actual precision/recall/abstention/latency with denominators.

[ ] T74 Review public naming/licenses/install path. Done: no unverified branding or model redistribution rights.

[ ] T75 Consider more languages and research features. Done: evidence-based priorities; no unsupported feature claims.

First task to perform

T00 through T04, then T10. Keep the first iteration small enough to inspect. The first useful checkpoint is unsafe/safe/unknown ratio analysis, before every other promised feature.